Solution: Longest Arithmetic Subsequence

Problem Statement

Given an array `nums` containing positive integers, return the maximum length of a subsequence that forms an `arithmetic progression`.

- A `subsequence` is an array that can be formed from `nums` by deleting 0 or more elements without changing the order of the remaining elements.
- An arithmetic progression (AP) is a sequence of numbers in which the difference between any two consecutive elements is constant. In short, `seq[i + 1] - seq[i]` should be same for all `0 <= i < seq.length - 2`.

Examples

- **Example 1**:
  - **Input**: `[8, 12, 6, 4, 2]`
  - **Expected Output**: `4`
  - **Justification**: The subsequence `[8, 6, 4, 2]` forms the longest arithmetic subsequence with a common difference of **-2**, thus the longest arithmetic subsequence has a length **4.**
- **Example 2**:
  - **Input**: `[5, 10, 15, 7, 11, 5]`
  - ****Expected Output: `3`
  - Justification:** The subsequence [**5, 10, 15**] forms the longest arithmetic subsequence with a common difference of **5**.
- **Example 3**:
  - **Input**: `[1, 7, 10, 15, 27, 29]`
  - **Expected Output**: `3`
  - **Justification**: The subsequence [**1, 15, 29**] forms the longest arithmetic subsequence with a common difference of **14**.

Solution

To solve this problem, a dynamic programming approach is most effective. The key idea is to use a hash table to store the lengths of the longest arithmetic subsequences ending at each element, indexed by their differences. This method allows for an efficient update of subsequences as the array is iterated through.

By considering each pair of elements in the array, we can update the hash table entries to reflect the longest subsequence lengths observed so far. This approach is favored because it enables the algorithm to consider all possible pairs of elements and their potential to extend existing arithmetic subsequences without needing to explicitly track each subsequence.

Step-by-step Algorithm

- Initialize a list of hash maps, where each hash map corresponds to an element in the input array. Each map will track the lengths of arithmetic subsequences ending with that element, keyed by their common differences.
- Iterate through each pair of elements in the array. For each pair `(i, j)` where `i < j`, calculate the difference `diff = array[i] - array[j]`.
- For each difference, update the hash map corresponding to `array[i]` to reflect the longest subsequence ending at `i` with difference `diff`. This is done by checking the length of the longest subsequence ending at `j` with the same difference and incrementing it by one.
- If no such subsequence exists for `array[j]`, initialize it with a length of 2 (since at least two elements are needed to form a subsequence).
- Keep track of the maximum length found during this process.
- Return the maximum length as the result.

Algorithm Walkthrough

Consider the input `[1, 7, 10, 15, 27, 29]`.

Initialization

- Initialize `dp` as an array of hash maps, one for each element in `A`.
- `result` is initialized to 2, the minimum possible length of an arithmetic sequence.

Processing

Iteration `i = 1` (**Element** `7`):

- **j = 0**: Compare `7` with `1`.
  - Difference `diff = 6`. Update `dp[1][6] = 2`.
  - HashMap for `i=1`: `{6: 2}`.
- `maxLength: 2`

Iteration `i = 2` (**Element** `10`):

- **j = 0**: Compare `10` with `1`.
  - Difference `diff = 9`. Update `dp[2][9] = 2`.
  - HashMap for `i=2` after `j=0`: `{9: 2}`.
- **j = 1**: Compare `10` with `7`.
  - Difference `diff = 3`. Update `dp[2][3] = 2`.
  - HashMap for `i=2` after `j=1`: `{9: 2, 3: 2}`.
- `maxLength: 2`

Iteration `i = 3` (**Element** `15`):

- **j = 0**: Compare `15` with `1`.
  - Difference `diff = 14`. Update `dp[3][14] = 2`.
  - HashMap for `i=3` after `j=0`: `{14: 2}`.
- **j = 1**: Compare `15` with `7`.
  - Difference `diff = 8`. Update `dp[3][8] = 2`.
  - HashMap for `i=3` after `j=1`: `{14: 2, 8: 2}`.
- **j = 2**: Compare `15` with `10`.
  - Difference `diff = 5`. Update `dp[3][5] = 2`.
  - HashMap for `i=3` after `j=2`: `{14: 2, 8: 2, 5: 2}`.
- `maxLength: 2`

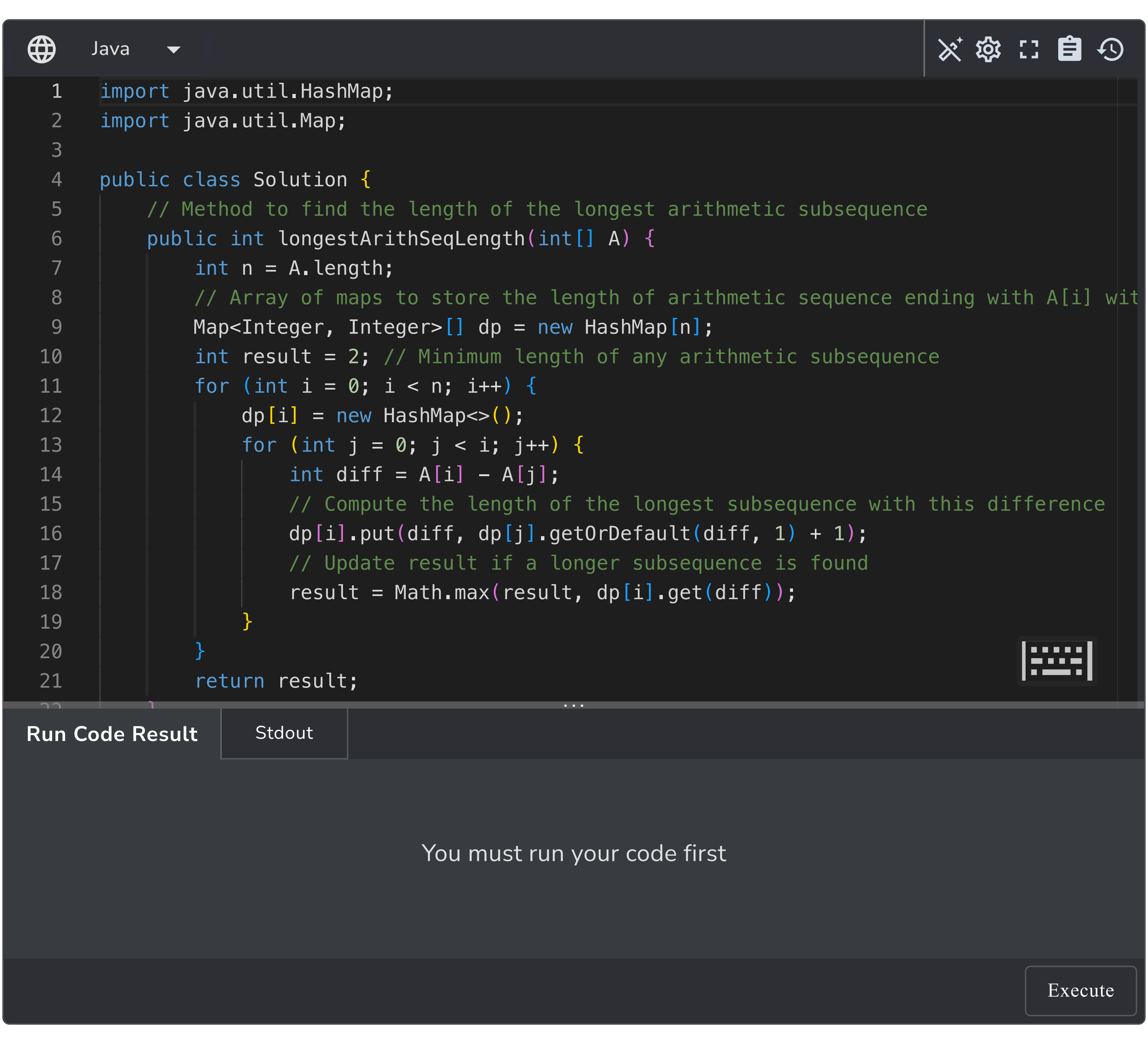
Iteration `i = 4` (**Element** `27`):

- **j = 0**: Compare `27` with `1`.
  - Difference `diff = 26`. Update `dp[4][26] = 2`.
  - HashMap for `i=4` after `j=0`: `{26: 2}`.
- **j = 1**: Compare `27` with `7`.
  - Difference `diff = 20`. Update `dp[4][20] = 2`.
  - HashMap for `i=4` after `j=1`: `{26: 2, 20: 2}`.
- **j = 2**: Compare `27` with `10`.
  - Difference `diff = 17`. Update `dp[4][17] = 2`.
  - HashMap for `i=4` after `j=2`: `{26: 2, 20: 2, 17: 2}`.
- **j = 3**: Compare `27` with `15`.
  - Difference `diff = 12`. Update `dp[4][12] = 2`.
  - HashMap for `i=4` after `j=3`: `{26: 2, 20: 2, 17: 2, 12: 2}`.
- `maxLength: 2`

Iteration `i = 5` (**Element** `29`):

- **j = 0**: Compare `29` with `1`.
  - Difference `diff = 28`. Update `dp[5][28] = 2`.
  - HashMap for `i=5` after `j=0`: `{28: 2}`.
- **j = 1**: Compare `29` with `7`.
  - Difference `diff = 22`. Update `dp[5][22] = 2`.
  - HashMap for `i=5` after `j=1`: `{28: 2, 22: 2}`.
- **j = 2**: Compare `29` with `10`.
  - Difference `diff = 19`. Update `dp[5][19] = 2`.
  - HashMap for `i=5` after `j=2`: `{28: 2, 22: 2, 19: 2}`.
- **j = 3**: Compare `29` with `15`.
  - Difference `diff = 14`. Update `dp[5][14] = dp[3].getOrDefault(14, 1) + 1 = 3`.
  - This update extends the sequence `[1, 15]` to `[1, 15, 29]`, with a common difference of `14`.
  - HashMap for `i=5` after `j=3`: `{28: 2, 22: 2, 19: 2, 14: 3}`.
- **j = 4**: Compare `29` with `27`.
  - Difference `diff = 2`. Update `dp[5][2] = 2`.
  - HashMap for `i=5` after `j=4`: `{28: 2, 22: 2, 19: 2, 14: 3, 2: 2}`.
- `maxLength: 3`
- The final `result` is updated to `3` when processing `i = 5` and `j = 3`, identifying the longest arithmetic subsequence `[1, 15, 29]` with a difference of `14`.

Code



```java
import java.util.HashMap;
import java.util.Map;

public class Solution {
    // Method to find the length of the longest arithmetic subsequence
    public int longestArithSeqLength(int[] A) {
        int n = A.length;
        // Array of maps to store the length of arithmetic sequence ending with A[i] wit
        Map<Integer, Integer>[] dp = new HashMap[n];
        int result = 2; // Minimum length of any arithmetic subsequence
        for (int i = 0; i < n; i++) {
            dp[i] = new HashMap<>();
            for (int j = 0; j < i; j++) {
                int diff = A[i] - A[j];
                // Compute the length of the longest subsequence with this difference
                dp[i].put(diff, dp[j].getOrDefault(diff, 1) + 1);
                // Update result if a longer subsequence is found
                result = Math.max(result, dp[i].get(diff));
            }
        }
        return result;
```

Complexity Analysis

Time Complexity

- $O(n^2)$: The primary operation in the algorithm is a double loop over the input array of size (n), where (n) is the length of the input array. For each element, the algorithm calculates the difference with every other element that comes before it, updating the hash map accordingly. This nested iteration leads to a quadratic time complexity.

Space Complexity

- $O(n^2)$: The algorithm maintains a hash map for each of the (n) elements in the input array, where each hash map can potentially store up to (n-1) key-value pairs (representing the arithmetic differences and their corresponding lengths). This results in a space complexity that is quadratic in the worst case.

← Previous
Longest Arithmetic Subsequence (Medium)

Next →
Matchsticks To Square (Medium)

☐ Mark as Completed